

第二个月详细指导书

AI-first LQCD Meson DA Analysis Agent

从可复现 CLI 到 Workflow + Analysis Coordinator 原型

For Graduate Student Training

第二个月核心目标

第二个月的目标不是继续写零散脚本，而是把第一个月已经能跑通的 golden example CLI，升级成一个尽可能自动执行、可复现、可追踪、可增量重跑、可初步判断 fit 质量的科研 AI-agent 原型。学生需要真正动手实现 Snakemake workflow、Analysis Engineer wrapper、Fit Strategy Postdoc、Critical Referee、rule-based Analysis Coordinator 和 decision log，并让这些环节尽量由 AI 负责完成，而不是只理解概念。

目录

1	第二个月在整个项目中的位置	4
2	本月最终交付物	4
3	推荐目录结构	5
4	每周安排总览	6
5	Week 5: 把 Month 1 CLI 升级为 Snakemake Workflow	8
5.1	本周目标	8
5.2	本周学习内容	8
5.3	Day 1: 整理 Month 1 的命令流	8
5.4	Day 2: 创建最小 Snakefile	9
5.5	Day 3: 测试 dry-run 和真实运行	10
5.6	Day 4: 拆分 rules 文件	10
5.7	Day 5: 加 Snakemake smoke test	11
5.8	Week 5 Codex Prompt	11
5.9	Week 5 命令示例	12
5.10	Week 5 常见错误	12
5.11	Week 5 导师检查问题	12
5.12	Week 5 验收标准	12

6	Week 6: 强化 Analysis Engineer Agent Wrapper	14
6.1	本周目标	14
6.2	Analysis Engineer 的职责边界	14
6.3	Day 1: 定义 Analysis Engineer task schema	15
6.4	Day 2: 实现 Analysis Engineer run	15
6.5	Day 3: 加入安全规则	16
6.6	Day 4: 加入 output validation	17
6.7	Day 5: Analysis Engineer tests	17
6.8	Week 6 Codex Prompt	17
6.9	Week 6 命令示例	18
6.10	Week 6 常见错误	18
6.11	Week 6 导师检查问题	19
6.12	Week 6 验收标准	19
7	Week 7: Fit Strategy Postdoc 与 Critical Referee	20
7.1	本周目标	20
7.2	Fit Strategy Postdoc 应该输出什么	20
7.3	Day 1: 统一 summary.csv 字段	20
7.4	Day 2: 实现 fit diagnostics metrics	21
7.5	Day 3: 实现 Fit Strategy Postdoc	21
7.6	Day 3.5: 实现 Matrix Element Analyst (bare ME 中间分析)	22
7.7	Day 4: 实现 Critical Referee	22
7.8	Day 5: 测试 Fit Strategy Postdoc 和 Critical Referee	23
7.9	Week 7 Codex Prompt	23
7.10	Week 7 命令示例	24
7.11	Week 7 常见错误	24
7.12	Week 7 导师检查问题	24
7.13	Week 7 验收标准	25
8	Week 8: Rule-based Analysis Coordinator、Decision Log 与 Month 2 Demo	26
8.1	本周目标	26
8.2	Analysis Coordinator 输入输出	26
8.3	Day 1: 定义 Analysis Coordinator schema	26
8.4	Day 2: 实现 rule-based Analysis Coordinator	27
8.5	Day 3: 生成 coordinator report	27
8.6	Day 4: 把 Analysis Coordinator 加入 Snakemake	28
8.7	Day 5: Month 2 demo 与总结	29
8.8	Week 8 Codex Prompt	29
8.9	Week 8 命令示例	30
8.10	Week 8 常见错误	30

8.11 Week 8 导师检查问题	31
8.12 Week 8 验收标准	31
9 第二个月最终运行命令	32
10 第二个月学生需要提交的周报模板	32
11 导师第二个月检查清单	33
11.1 Workflow 检查	33
11.2 Analysis Engineer 检查	33
11.3 Fit Strategy Postdoc、Matrix Element Analyst 和 Critical Referee 检查	34
11.4 Analysis Coordinator 检查	34
12 第二个月常见失败模式	34
12.1 失败模式 1: Snakefile 里写了太多 Python 分析逻辑	34
12.2 失败模式 2: Analysis Engineer 变成了 Analysis Coordinator	34
12.3 失败模式 3: Critical Referee 只重复 Fit Strategy Postdoc 的结论	34
12.4 失败模式 4: Decision log 没有 evidence	34
12.5 失败模式 5: Codex 一次改太多	34
13 Month 2 完成标准	35

1 第二个月在整个项目中的位置

第一个月的重点是已分析流程变成可运行、可配置、可测试的 Python CLI，并建立 schema、pytest、Correlator Analyst 和最小 Analysis Engineer wrapper。第二个月的重点是把模块组织成更接近真实科研系统的 workflow 和 agent pipeline，并尽量把重复、机械、可检查的工作交给 AI 自动完成，只把少数关键判断留给人类。

本月的关键词是：

- **Snakemake workflow**: 用于管理真实分析流程中的文件依赖、增量重跑和批量任务。
- **Analysis Engineer Agent**: 负责执行 workflow、检查输入输出、记录运行日志和错误。
- **Fit Strategy Postdoc**: 读取 fit scan 结果，提取稳定性、模型依赖和统计质量指标。
- **Critical Referee**: 专门寻找问题，而不是直接接受结果。
- **Matrix Element Analyst**: 检查 bare matrix element、 z -dependence、 P_z -dependence、real/imaginary symmetry 等中间物理量。
- **Rule-based Analysis Coordinator**: 根据结构化指标和 reviewer 意见给出 fit recommendation、risk level 和 Human Approval Gate 标记。
- **decision_log.json**: 记录为什么推荐某个 fit，为什么拒绝某些结果，以及是否需要人工审批。

本月完成的 agent 包括 Analysis Engineer (强化)、Fit Strategy Postdoc、Matrix Element Analyst、Critical Referee 和 rule-based Analysis Coordinator / PI。

本月完成后，系统应该可以做到：

```
# 1. 用 Snakemake 跑完整 golden example workflow
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1

# 2. 用 Analysis Engineer 包装运行过程并生成 structured task output
python -m lqcd_agent.worker.run --task configs/tasks/run_l64c64a076_analysis_engineer.yaml

# 3. 用 Analysis Coordinator 检查 fit scan / ratio scan 结果并生成 decision log
python -m lqcd_agent.supervisor.run --case outputs/l64c64a076_m140/case_manifest.json
```

2 本月最终交付物

第二个月结束时，学生至少应提交以下内容：

交付物	说明
-----	----

workflow/Snakefile	能复现 golden example 的最小 Snakemake workflow。
workflow/rules/*.smk	按模块拆分的 workflow rules。
configs/l64c64a076_m140.yaml	golden example 的主配置文件。
src/lqcd_agent/analysis_engineer.py	Analysis Engineer wrapper (task schema + safe execution + output validation)。
src/lqcd_agent/analysis/FitStrategyPostdoc.py	读取 Pysummary 并生成 fit hypotheses。
src/lqcd_agent/analysis/MatrixElementAnalysis.py	检查 bare ME、 z/P_z -dependence。
src/lqcd_agent/critical_referee/critical_referee.py	critical referee 检查 rules。
src/lqcd_agent/analysis_coordinator/	Coordinator / PI: 综合决策 + risk level assignment。
outputs/.../fit_strategy_recommendation.json	策略推荐 (含物理理由)。
outputs/.../fit_candidates.csv	候选 fit 对比表。
outputs/.../matrix_element_diagnosis.json	bare ME 诊断结果。
outputs/.../critical_referee_report.json	Critical Referee 完整审查报告。
outputs/.../decision_log.json	结构化决策日志 (含 risk level、reasons、evidence)。
outputs/.../coordinator_summary.md	Coordinator 人类可读摘要。
outputs/.../next_action_plan.json	下一步行动计划 (含 targeted rerun)。
tests/test_snakemake_smoke.py	workflow smoke test。
tests/test_analysis_engineer.py	Analysis Engineer 安全 + 输出测试。
tests/test_fit_strategy_postdoc.py	Fit Strategy Postdoc 测试 (3 种 case)。
tests/test_critical_referee.py	Critical Referee 测试 (3 种 case)。
tests/test_analysis_coordinator.py	Coordinator 规则测试。
docs/month2_report.md	Month 2 总结报告 (含 demo 运行说明)。

3 推荐目录结构

第二个月结束时, repo 建议扩展为下面结构。学生不一定一次性全部创建, 而是按每周任务逐步增加。

```
lqcd-meson-da-agent/
  configs/
    l64c64a076_m140.yaml
  tasks/
    run_l64c64a076_analysis_engineer.yaml
  workflow/
    Snakefile
    config.yaml
    rules/
      audit.smk
      run_analysis.smk
      collect_summary.smk
      coordinator.smk
```

```
src/lqcd_agent/  
  analysis/  
    fit_strategy_postdoc.py  
    metrics.py  
    summary_io.py  
  reviewer/  
    critical_referee.py  
    checks.py  
  supervisor/  
    schema.py  
    rules.py  
    run.py  
    report.py  
  worker/  
    schema.py  
    run.py  
    safety.py  
    validate_outputs.py  
  tests/  
    test_snakemake_smoke.py  
    test_analysis_engineer.py  
    test_fit_strategy_postdoc.py  
    test_critical_referee.py  
    test_analysis_coordinator.py  
  docs/  
    month2_report.md  
    decision_log_spec.md  
  outputs/  
    164c64a076_m140/  
      case_manifest.json  
      summary.csv  
      fit_strategy_report.json  
      critical_referee_report.json  
      decision_log.json  
      coordinator_summary.md
```

4 每周安排总览

周次	主题	核心产出
Week 5	Snakemake workflow	把 Month 1 CLI 包装成可复现 workflow，支持增量重跑。

Week 6	Analysis Engineer Agent 强化	task schema、safe run、output validation、error handling。
Week 7	Fit Strategy Postdoc + Critical Referee	从 fit scan/summary 中提取稳定性指标，并系统性寻找风险。
Week 8	Rule-based Analysis Coordinator + Decision Log	生成 recommendation、risk level、Human Approval Gate flag 和 coordinator report。

5 Week 5: 把 Month 1 CLI 升级为 Snakemake Workflow

5.1 本周目标

本周目标是把第一个月已经能手动运行的 CLI 流程，组织成 Snakemake workflow。学生要理解：Snakemake 不是 agent，也不是替代 Python 分析代码；它负责把可运行的命令变成可复现的依赖图。

本周结束时，学生应该能运行：

```
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1
```

并自动生成：

```
outputs/l64c64a076_m140/  
  audit_report.json  
  result.json  
  summary.csv  
  plots/  
  case_manifest.json
```

5.2 本周学习内容

学生需要学习到能完成任务即可，不需要深入所有高级功能。

1. Snakemake 的基本概念：rule、input、output、shell、params、config。
2. 文件依赖关系：为什么 output 存在且比 input 新时不需要重跑。
3. workflow 和 CLI 的关系：rule 只调用已有 CLI，不在 Snakefile 中写复杂分析逻辑。
4. 如何用 --dry-run 检查计划。
5. 如何用 --rerun-incomplete 处理失败任务。

5.3 Day 1: 整理 Month 1 的命令流

第一天不要急着写 Snakemake。先把 Month 1 手动命令整理清楚。

学生需要写一个文件：

```
docs/month1_manual_pipeline.md
```

内容包括：

```
# Step 1: audit input data  
python -m lqcd_agent.audit.run --config configs/l64c64a076_m140.yaml  
  
# Step 2: run analysis CLI  
python -m lqcd_agent.analysis.run --config configs/l64c64a076_m140.yaml
```



```
# Step 3: collect summary
```

```
python -m lqcd_agent.analysis.collect_summary --case outputs/l64c64a076_m140/  
case_manifest.json
```

如果实际命令名与上面不同，以当前 repo 已有命令为准。关键是：每一步的输入、输出、命令必须写清楚。

5.4 Day 2: 创建最小 Snakefile

创建：

```
workflow/Snakefile
```

最小版本可以先写成：

```
configfile: "configs/l64c64a076_m140.yaml"
```



```
CASE = config["case_id"]  
OUTDIR = config["output_dir"]
```



```
rule all:  
    input:  
        f"{OUTDIR}/case_manifest.json",  
        f"{OUTDIR}/summary.csv"
```



```
rule audit_data:  
    input:  
        config="configs/l64c64a076_m140.yaml"  
    output:  
        f"{OUTDIR}/audit_report.json"  
    shell:  
        "python -m lqcd_agent.audit.run --config {input.config} --output {output}"
```



```
rule run_analysis:  
    input:  
        config="configs/l64c64a076_m140.yaml",  
        audit=f"{OUTDIR}/audit_report.json"  
    output:  
        result=f"{OUTDIR}/result.json",  
        summary=f"{OUTDIR}/summary.csv"  
    shell:  
        "python -m lqcd_agent.analysis.run --config {input.config}"
```



```
rule make_case_manifest:  
    input:  
        audit=f"{OUTDIR}/audit_report.json",  
        result=f"{OUTDIR}/result.json",
```

```
summary=f"{OUTDIR}/summary.csv"
output:
  f"{OUTDIR}/case_manifest.json"
shell:
  "python -m lqcd_agent.analysis.make_manifest --config configs/l64c64a076_m140.
  yaml --output {output}"
```

如果当前项目还没有这些 CLI，学生需要用 Codex 补齐最小版本，但不能把复杂逻辑写进 Snakefile。

5.5 Day 3: 测试 dry-run 和真实运行

先运行 dry-run:

```
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1 --
dry-run
```

检查输出是否显示预期 rules。

然后真实运行:

```
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1 --
rerun-incomplete
```

运行后检查:

```
ls -lh outputs/l64c64a076_m140/
cat outputs/l64c64a076_m140/case_manifest.json
head outputs/l64c64a076_m140/summary.csv
```

5.6 Day 4: 拆分 rules 文件

如果 Snakefile 开始变长，就拆成:

```
workflow/Snakefile
workflow/rules/audit.smk
workflow/rules/run_analysis.smk
workflow/rules/collect_summary.smk
```

主 Snakefile:

```
configfile: "configs/l64c64a076_m140.yaml"

CASE = config["case_id"]
OUTDIR = config["output_dir"]

include: "workflow/rules/audit.smk"
include: "workflow/rules/run_analysis.smk"
include: "workflow/rules/collect_summary.smk"
```

```
rule all:
    input:
        f"{OUTDIR}/case_manifest.json",
        f"{OUTDIR}/summary.csv"
```

5.7 Day 5: 加 Snakemake smoke test

创建:

```
tests/test_snakemake_smoke.py
```

测试目标不是完整大规模运行，而是用一个 tiny config 跑通 workflow。
建议让 Codex 写一个最小 smoke test:

```
def test_snakemake_dry_run():
    # run snakemake --dry-run on a tiny test config
    # assert return code == 0
    pass
```

5.8 Week 5 Codex Prompt

We are in Month 2 of an AI-first LQCD meson DA analysis-agent project.
Month 1 produced a config-driven Python CLI for the golden example.
Now we need to wrap the existing CLI commands into a minimal Snakemake workflow.

Task:

Create `workflow/Snakefile` and, if useful, `workflow/rules/\*.smk`.

Requirements:

- The workflow should use `configs/l64c64a076\_m140.yaml` as the main config.
- It should call existing Python CLI commands; do not put analysis logic inside the Snakefile.
- It should produce at least:
 - audit_report.json
 - result.json
 - summary.csv
 - case_manifest.json
- Add a dry-run instruction to `docs/month2\_report.md`.
- Add a pytest smoke test that runs Snakemake in dry-run mode on a tiny config.

Constraints:

- Do not change the analysis algorithms.
- Do not hard-code absolute machine-specific paths.
- Do not overwrite raw data.
- Keep the first workflow simple and readable.

After implementation, explain:

- how to run the workflow;
- which files are inputs and outputs;
- how to test it.

5.9 Week 5 命令示例

```
# dry-run
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1 --dry-run
# 正式运行
snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.yaml --cores 1 --rerun-incomplete
# 检查输出
ls -lh outputs/l64c64a076_m140/
# smoke test
pytest tests/test_snakemake_smoke.py -q
```

5.10 Week 5 常见错误

- Snakefile 里写复杂分析逻辑:Snakefile 只负责调度 rule。分析逻辑保留在 src/lqcd_agent/ 中通过 CLI 调用。
- rule 的 input/output 不完整: 缺少输入输出声明会导致增量重跑失效。
- 没有 dry-run 就直接正式跑: 先用 --dry-run 检查依赖图。

5.11 Week 5 导师检查问题

- snakemake --dry-run 是否展示正确的依赖图?
- Snakefile 中是否有复杂分析逻辑 (应只调用 CLI)?
- 修改一个输入后, 是否只重跑必要的下游步骤?

5.12 Week 5 验收标准

导师验收时检查:

- snakemake --dry-run 可以成功。
- 正常运行可以生成 expected outputs。
- Snakefile 中没有复杂分析逻辑, 只调用 CLI。
- 输出目录结构清楚。

- 学生能解释每个 rule 的 input/output。
- Git 至少有 3 个小 commit。

6 Week 6: 强化 Analysis Engineer Agent Wrapper

6.1 本周目标

本周目标是把 workflow 的执行封装成 Analysis Engineer Agent。Analysis Engineer 不负责决定物理结果，而负责安全、可复现、可追踪地执行任务。

本周结束时，学生应该能运行：

```
python -m lqcd_agent.worker.run --task configs/tasks/run_l64c64a076_analysis_engineer.yaml
```

并生成：

```
outputs/l64c64a076_m140/analysis_engineer_output.json
outputs/l64c64a076_m140/analysis_engineer_run.log
```

6.2 Analysis Engineer 的职责边界

Analysis Engineer 可以做：

- 读取 task config。
- 检查输入文件是否存在。
- 调用 Snakemake 或 Python CLI。
- 检查输出文件是否生成。
- 记录命令、返回码、运行时间和错误。
- 生成 structured output。

Analysis Engineer 不可以做：

- 选择 central fit。
- 判断 systematic uncertainty。
- 覆盖 accepted result。
- 修改 raw data。
- 运行不可追踪的大范围 shell 命令。

6.3 Day 1: 定义 Analysis Engineer task schema

创建:

```
src/lqcd_agent/analysis_engineer/schema.py
configs/tasks/run_l64c64a076_analysis_engineer.yaml
```

YAML 示例:

```
task_id: run_l64c64a076_m140_v1
task_type: run_snakemake_workflow
case_id: l64c64a076_m140
config_path: configs/l64c64a076_m140.yaml
snakefile: workflow/Snakefile
output_dir: outputs/l64c64a076_m140
cores: 1
allow_overwrite: false
expected_outputs:
  - outputs/l64c64a076_m140/audit_report.json
  - outputs/l64c64a076_m140/result.json
  - outputs/l64c64a076_m140/summary.csv
  - outputs/l64c64a076_m140/case_manifest.json
```

Pydantic schema 示例:

```
from pydantic import BaseModel, Field
from typing import Literal

class AnalysisEngineerTask(BaseModel):
    task_id: str
    task_type: Literal["run_snakemake_workflow", "run_python_cli"]
    case_id: str
    config_path: str
    output_dir: str
    allow_overwrite: bool = False
    expected_outputs: list[str] = Field(default_factory=list)
    snakefile: str | None = None
    cores: int = 1
```

6.4 Day 2: 实现 Analysis Engineer run

创建:

```
src/lqcd_agent/analysis_engineer/run.py
```

核心逻辑:

1. 读取 task YAML。

2. 用 Pydantic validate。
3. 检查 config 是否存在。
4. 如果 output 已存在且 allow_overwrite=false, 拒绝覆盖或要求使用新的 output dir。
5. 组装 Snakemake 命令。
6. 用 subprocess.run 执行。
7. 捕获 stdout/stderr/return code。
8. 检查 expected outputs 是否存在。
9. 写 worker_output.json。

Analysis Engineer output 示例:

```
{
  "task_id": "run_l64c64a076_m140_v1",
  "case_id": "l64c64a076_m140",
  "status": "success",
  "command": "snakemake -s workflow/Snakefile --configfile configs/l64c64a076_m140.
    yaml --cores 1",
  "return_code": 0,
  "outputs": {
    "summary_csv": "outputs/l64c64a076_m140/summary.csv",
    "case_manifest": "outputs/l64c64a076_m140/case_manifest.json"
  },
  "missing_outputs": [],
  "warnings": [],
  "errors": []
}
```

6.5 Day 3: 加入安全规则

创建:

```
src/lqcd_agent/analysis_engineer/safety.py
```

至少实现以下规则:

- 禁止 task config 中出现 raw data output path。
- 禁止在 Analysis Engineer 中执行 `rm -rf`。
- 默认不覆盖已有 output directory。
- 命令必须来自白名单, 例如 `snakemake` 或 `python -m lqcd_agent.*`。
- 所有 command 必须写入 log。

6.6 Day 4: 加入 output validation

创建:

```
src/lqcd_agent/analysis_engineer/validate_outputs.py
```

检查:

- expected outputs 是否存在。
- summary.csv 是否非空。
- result.json 是否能被 JSON 读取。
- case_manifest.json 是否包含 case id、config path、output paths。

6.7 Day 5: Analysis Engineer tests

测试文件:

```
tests/test_analysis_engineer.py
```

至少测试:

- valid task 可以通过 schema validation。
- missing config 会失败且错误清楚。
- expected output 缺失会进入 missing_outputs。
- allow_overwrite=false 时不覆盖已有 output。
- unsafe command 被拒绝。

6.8 Week 6 Codex Prompt

We are building the Analysis Engineer Agent for an AI-first LQCD meson DA analysis system.

The Analysis Engineer should execute an existing Snakemake workflow safely and produce structured output.

Task:

Implement Analysis Engineer task schema, runner, safety checks, and output validation.

Files to create or edit:

- src/lqcd_agent/analysis_engineer/schema.py
- src/lqcd_agent/analysis_engineer/run.py
- src/lqcd_agent/analysis_engineer/safety.py
- src/lqcd_agent/analysis_engineer/validate_outputs.py
- configs/tasks/run_l64c64a076_analysis_engineer.yaml

```
- tests/test_analysis_engineer.py
```

Requirements:

- Use Pydantic for AnalysisEngineerTask and AnalysisEngineerOutput schemas.
- Read a YAML task file from CLI argument `--task`.
- Support task_type = run_snakemake_workflow.
- Construct a Snakemake command from the task fields.
- Capture return code, stdout, stderr, and runtime.
- Check expected output files after execution.
- Write analysis_engineer_output.json to the output directory.
- Refuse unsafe operations and avoid overwriting by default.

Constraints:

- Analysis Engineer must not decide central physics results.
- Analysis Engineer must not modify raw data.
- Do not use broad shell commands.
- Keep implementation simple and testable.

Tests:

- valid task schema;
- missing config path;
- missing expected output detection;
- unsafe command rejection;
- overwrite protection.

6.9 Week 6 命令示例

```
# 运行 Analysis Engineer task
python -m lqcd_agent.analysis_engineer.run --task configs/tasks/
    run_l64c64a076_analysis_engineer.yaml
# 检查输出
cat outputs/l64c64a076_m140/analysis_engineer_output.json
# 运行测试
pytest tests/test_analysis_engineer.py -q
```

6.10 Week 6 常见错误

- **Analysis Engineer 变成了 Analysis Coordinator**: Analysis Engineer 只能执行、检查、记录。所有 recommendation 必须放到 Analysis Coordinator。
- **没有 overwrite protection**: 默认不覆盖已有输出; 必须显式设置 allow_overwrite=true。
- **命令不是白名单**: Analysis Engineer 执行的命令必须来自白名单。

6.11 Week 6 导师检查问题

- Analysis Engineer 是否有 Pydantic task schema?
- 是否有 overwrite protection 和 output validation?
- 失败时是否生成清楚的错误信息 (不是 silent fail)?
- 学生能否解释 Analysis Engineer 为什么不能决定 central fit?

6.12 Week 6 验收标准

- Analysis Engineer 可以调用 Snakemake workflow。
- Analysis Engineer 生成 `worker_output.json`。
- 失败时不会 silent fail。
- 安全规则有测试覆盖。
- 学生能解释 Analysis Engineer 为什么不能决定 central fit。

7 Week 7: Fit Strategy Postdoc 与 Critical Referee

7.1 本周目标

本周开始进入“分析判断”的前半部分：先不让 Analysis Coordinator 直接做最终推荐，而是先建立两个中间角色。

- **Fit Strategy Postdoc**: 从 summary.csv / fit scan 中提取稳定性和质量指标。
- **Critical Referee**: 专门寻找潜在问题，提出 warnings 和 failure modes。

这种设计借鉴了 advanced multi-agent practice 中的 specialized role 和 critic/reviewer 思路。一个 agent 不应该既生成结果又完全相信自己。科研分析尤其需要一个“挑错者”。

7.2 Fit Strategy Postdoc 应该输出什么

输入：

```
outputs/l64c64a076_m140/summary.csv
outputs/l64c64a076_m140/result.json
```

输出：

```
outputs/l64c64a076_m140/fit_strategy_report.json
```

建议结构：

```
{
  "case_id": "l64c64a076_m140",
  "n_candidate_fits": 24,
  "models": ["1state", "2state"],
  "best_chi2_candidates": [...],
  "stability_metrics": {
    "E0_tmin_slope": 0.003,
    "E0_tmin_max_shift_sigma": 0.8,
    "model_difference_sigma": 1.2
  },
  "recommended_candidates_for_review": [
    "2state_tmin4_tmax15",
    "2state_tmin5_tmax15"
  ]
}
```

7.3 Day 1: 统一 summary.csv 字段

Fit Strategy Postdoc 能否工作，首先取决于 summary.csv 是否标准化。

建议字段：

```
case_id,observable,model,tmin,tmax,param,value,error,chi2_dof,p_value,status,notes
```

如果现有 summary.csv 字段不同，学生应先写 adapter，而不是到处改已有代码。

创建：

```
src/lqcd_agent/analysis/summary_io.py
```

功能：

- 读取 summary.csv。
- 检查 required columns。
- 返回 pandas DataFrame。
- 对缺失列给出清楚错误。

7.4 Day 2: 实现 fit diagnostics metrics

创建：

```
src/lqcd_agent/analysis/metrics.py
```

至少实现：

- **chi2 quality**: 是否 χ^2/dof 过大。
- **tmin stability**: 相邻 tmin 的结果是否在误差内稳定。
- **model dependence**: 1-state 与 2-state 是否显著不一致。
- **fit failure rate**: fit scan 中失败比例。

简单 sigma difference:

$$\Delta_\sigma = \frac{|x_1 - x_2|}{\sqrt{\sigma_1^2 + \sigma_2^2}}. \quad (1)$$

学生需要理解：这个指标不是最终物理判断，只是 reviewer 和 supervisor 的输入。

7.5 Day 3: 实现 Fit Strategy Postdoc

创建：

```
src/lqcd_agent/analysis/fit_strategy_postdoc.py
```

CLI 示例：

```
python -m lqcd_agent.analysis.fit_analyst \
--summary outputs/l64c64a076_m140/summary.csv \
--output outputs/l64c64a076_m140/fit_strategy_report.json
```

Fit Strategy Postdoc 不做最终选择，只做结构化诊断。

7.6 Day 3.5: 实现 Matrix Element Analyst (bare ME 中间分析)

Matrix Element Analyst 负责检查 bare matrix element 的中间物理量。创建:

```
src/lqcd_agent/analysis/matrix_element_analyst.py
```

检查内容:

- 读取 bare ME 输出和 ratio fit 结果;
- 检查 z -dependence 是否平滑 (标记非物理振荡);
- 检查 P_z -dependence 是否合理 (标记异常趋势);
- 检查 real / imaginary symmetry;
- 检查 local matrix element normalization ($z = 0$ 是否接近 1);
- 输出 matrix_element_diagnostics.json 和 bare_me_summary.csv。

7.7 Day 4: 实现 Critical Referee

创建:

```
src/lqcd_agent/critical_referee/critical_referee.py
```

输入:

```
fit_strategy_report.json  
summary.csv
```

输出:

```
critical_referee_report.json
```

输出示例:

```
{  
  "case_id": "l64c64a076_m140",  
  "risk_flags": [  
    {  
      "flag": "model_dependence",  
      "severity": "medium",  
      "reason": "1-state and 2-state E0 differ by 1.8 sigma"  
    },  
    {  
      "flag": "tmin_instability",  
      "severity": "low",  
      "reason": "neighboring tmin shifts are below 1 sigma for selected 2-state  
        candidates"  
    }  
  ]  
}
```

```

],
"questions_for_supervisor": [
    "Should 1-state results be used only as a diagnostic?",
    "Is the 2-state tmin=4 candidate stable enough for provisional recommendation?"
],
"human_attention_required": false
}

```

7.8 Day 5: 测试 Fit Strategy Postdoc 和 Critical Referee

测试:

```

tests/test_fit_strategy_postdoc.py
tests/test_critical_referee.py

```

至少构造 3 个 fake summary:

1. good stable case.
2. large chi2 case.
3. strong model dependence case.

7.9 Week 7 Codex Prompt

We are adding Fit Strategy Postdoc and Critical Referee modules to the LQCD meson DA analysis-agent project.

These modules should inspect fit scan results but should not make the final central-fit decision.

Task:

Implement standardized summary reading, fit diagnostics metrics, Fit Strategy Postdoc CLI, and Critical Referee CLI.

Files:

- src/lqcd_agent/analysis/summary_io.py
- src/lqcd_agent/analysis/metrics.py
- src/lqcd_agent/analysis/fit_strategy_postdoc.py
- src/lqcd_agent/critical_referee/critical_referee.py
- tests/test_fit_strategy_postdoc.py
- tests/test_critical_referee.py

Requirements:

- Read summary.csv with required columns:
 - case_id, observable, model, tmin, tmax, param, value, error, chi2_dof, p_value, status, notes

```

- Compute simple diagnostics:
  chi2 quality, tmin stability, model dependence, fit failure rate.
- Write fit_strategy_report.json.
- Critical Referee should read diagnostics and produce risk_flags with severity and
  reason.
- Add tests for stable case, large chi2 case, and model-dependence case.

Constraints:
- Do not select the final central result here.
- Do not hide failed fits.
- Keep metrics transparent and easy to explain.

```

7.10 Week 7 命令示例

```

# Fit Strategy Postdoc
python -m lqcd_agent.analysis.fit_strategy_postdoc \
  --summary outputs/l64c64a076_m140/summary.csv \
  --output outputs/l64c64a076_m140/fit_strategy_report.json
# Matrix Element Analyst
python -m lqcd_agent.analysis.matrix_element_analyst \
  --bare-me outputs/l64c64a076_m140/bare_me.csv \
  --output outputs/l64c64a076_m140/matrix_element_diagnostics.json
# Critical Referee
python -m lqcd_agent.critical_referee.check \
  --diagnostics-dir outputs/l64c64a076_m140/ \
  --output outputs/l64c64a076_m140/critical_referee_report.json
# 运行测试
pytest tests/test_fit_strategy_postdoc.py tests/test_critical_referee.py -q

```

7.11 Week 7 常见错误

- **Fit Strategy Postdoc 变成 grid search**: 它应该提出 2-5 个有物理动机的 candidate, 不是扫描所有 tmin。
- **Critical Referee 只重复 Fit Strategy Postdoc 的结论**: Critical Referee 的任务是找风险和漏洞, 提出具体的 requested check。
- **没有为 fake cases 写测试**: 至少需要 good/large chi2/strong model dependence 三种 case。

7.12 Week 7 导师检查问题

- Fit Strategy Postdoc 是否生成了 fit_strategy_report.json?
- Matrix Element Analyst 是否检查了 z-dependence 和 real/imag symmetry?

- Critical Referee 是否真的在挑问题（不只是复述结论）？是否有具体 evidence？
- 三种测试 case 是否覆盖？

7.13 Week 7 验收标准

- `fit_diagnostics.json` 能从 `summary.csv` 生成。
- `skeptic_review.json` 能指出至少三类问题。
- 学生能解释 Δ_σ 的含义。
- Critical Referee 不直接接受结果，而是输出 structured objections 和 requested checks。
- 测试覆盖 good、bad、ambiguous 三种情况。

8 Week 8: Rule-based Analysis Coordinator、Decision Log 与 Month 2 Demo

8.1 本周目标

本周把前面模块连接成一个初级 decision system。Analysis Coordinator 读取 Analysis Engineer、Fit Strategy Postdoc、Critical Referee 的输出, 生成 recommendation、risk level、Human Approval Gate flag 和 decision log。

注意: 这里的 Analysis Coordinator 仍然是 rule-based, 不是 LLM-based。原因是我们先要建立透明、可测试、可回放的决策机制。第三个月再考虑 LangGraph 和 LLM 接入。

8.2 Analysis Coordinator 输入输出

输入:

```
case_manifest.json
summary.csv
fit_strategy_report.json
critical_referee_report.json
analysis_engineer_output.json
```

输出:

```
decision_log.json
coordinator_summary.md
```

8.3 Day 1: 定义 Analysis Coordinator schema

创建:

```
src/lqcd_agent/analysis_coordinator/schema.py
```

建议 schema:

```
from pydantic import BaseModel
from typing import Literal

class DecisionReason(BaseModel):
    category: str
    message: str
    evidence: dict = {}

class FitRecommendation(BaseModel):
    case_id: str
    selected_candidate: str | None
    rejected_candidates: list[str]
    risk_level: Literal["low", "medium", "high"]
```

```

human_approval_required: bool
reasons: list[DecisionReason]
recommended_actions: list[str]

class DecisionLog(BaseModel):
    case_id: str
    decision_type: str
    inputs: dict
    recommendation: FitRecommendation
    created_by: str = "rule_based_supervisor_v1"

```

8.4 Day 2: 实现 rule-based Analysis Coordinator

创建:

```

src/lqcd_agent/analysis_coordinator/rules.py
src/lqcd_agent/analysis_coordinator/run.py

```

基础规则建议:

1. 如果 Analysis Engineer failed, Analysis Coordinator 不做 fit recommendation, 只建议 rerun/debug。
2. 如果所有 fit failed, risk=high, Human Approval Gate required。
3. 如果 χ^2/dof 普遍过大, risk=high。
4. 如果 model dependence 超过 2 sigma, risk=medium 或 high。
5. 如果 2-state fit 稳定而 1-state 不稳定, 可 provisional 推荐 2-state, 但要求 Human Approval Gate。
6. 如果 tmin stability 好、chi2 合理、model dependence 小, 则 risk=low。
7. 如果 reviewer 提出 high severity flag, 则 Human Approval Gate required。

8.5 Day 3: 生成 coordinator report

创建:

```

src/lqcd_agent/analysis_coordinator/report.py

```

生成 Markdown 报告:

```

# Analysis Coordinator Report: 164c64a076_m140

## Recommendation
Selected candidate: 2state_tmin4_tmax15
Risk level: medium

```

```
Human Approval Gate required: true

## Main reasons
1. 2-state fit shows better tmin stability.
2. 1-state fit has visible excited-state contamination.
3. Model difference is 1.6 sigma, so final approval is required.

## Risk flags from Critical Referee
- model_dependence: medium
- tmin_instability: low

## Recommended next actions
- Human should inspect fit_stability.pdf.
- If approved, add this case to accepted memory.
- If not approved, run targeted scan with larger tmin range.
```

8.6 Day 4: 把 Analysis Coordinator 加入 Snakemake

在 workflow 中加入:

```
rule fit_analyst:
    input:
        summary=f"{OUTDIR}/summary.csv"
    output:
        f"{OUTDIR}/fit_strategy_report.json"
    shell:
        "python -m lqcd_agent.analysis.fit_analyst --summary {input.summary} --output {output}"

rule critical_referee_review:
    input:
        diagnostics=f"{OUTDIR}/fit_strategy_report.json",
        summary=f"{OUTDIR}/summary.csv"
    output:
        f"{OUTDIR}/critical_referee_report.json"
    shell:
        "python -m lqcd_agent.viewer.skeptic --diagnostics {input.diagnostics} --summary {input.summary} --output {output}"

rule coordinator_decision:
    input:
        manifest=f"{OUTDIR}/case_manifest.json",
        summary=f"{OUTDIR}/summary.csv",
        diagnostics=f"{OUTDIR}/fit_strategy_report.json",
        review=f"{OUTDIR}/critical_referee_report.json"
```

```

output:
    decision=f"{OUTDIR}/decision_log.json",
    report=f"{OUTDIR}/coordinator_summary.md"
shell:
    "python -m lqcd_agent.supervisor.run --case {input.manifest} --summary {input.summary} --diagnostics {input.diagnostics} --review {input.review} --decision {output.decision} --report {output.report}"

```

8.7 Day 5: Month 2 demo 与总结

学生需要准备一个 10–15 分钟 demo。

Demo 必须包括：

1. 展示 Snakemake DAG (dry-run 输出)。
2. 运行 Analysis Engineer task 并展示 analysis_engineer_output.json。
3. 展示 summary.csv 和 fit_strategy_report.json。
4. 展示 matrix_element_diagnostics.json (含 z -dependence、 P_z -dependence plots)。
5. 展示 critical_referee_report.json (至少 1 个 major objection with evidence)。
6. 展示 decision_log.json (含 risk level、reasons、evidence)。
7. 展示 coordinator_summary.md 和 next_action_plan.json。
8. 解释 Analysis Coordinator 为什么给出该 recommendation (risk level 判据)。
9. 说明哪些地方仍需要 Human Approval Gate 审批 (high risk 场景)。

8.8 Week 8 Codex Prompt

We are implementing the rule-based Analysis Coordinator for the LQCD meson DA analysis -agent project.

The Analysis Coordinator reads all agent outputs (Analysis Engineer, Fit Strategy Postdoc, Matrix Element Analyst, Critical Referee), calculates risk level, and writes a structured decision log and a human-readable report.

Task:

Implement Analysis Coordinator schema, rules, CLI runner, and Markdown report generation.

Files:

- src/lqcd_agent/analysis_coordinator/schema.py
- src/lqcd_agent/analysis_coordinator/rules.py
- src/lqcd_agent/analysis_coordinator/run.py

```
- src/lqcd_agent/analysis_coordinator/report.py
- tests/test_analysis_coordinator.py
- update workflow rules to include fit_analyst, critical_referee_review, and
  coordinator_decision
```

Requirements:

```
- Read case_manifest.json, summary.csv, fit_strategy_report.json, and
  critical_referee_report.json.
- Produce decision_log.json following a Pydantic schema.
- Produce coordinator_summary.md.
- Assign risk_level = low, medium, or high.
- Set human_approval_required based on risk and reviewer flags.
- Include explicit reasons with evidence.
- Add tests for:
  1. good stable case;
  2. all fits failed;
  3. strong model dependence;
  4. high-severity reviewer flag.
```

Constraints:

```
- Analysis Coordinator may recommend but must not silently promote an accepted central
  result.
- Medium-risk and high-risk cases require Human Approval Gate.
- Keep rules transparent and easy to inspect.
```

8.9 Week 8 命令示例

```
# 运行 Analysis Coordinator
python -m lqcd_agent.analysis_coordinator.run \
  --case outputs/l64c64a076_m140/case_manifest.json
# 查看 decision log
cat outputs/l64c64a076_m140/decision_log.json
# 查看 summary
cat outputs/l64c64a076_m140/coordinator_summary.md
# 运行测试
pytest tests/test_analysis_coordinator.py -q
```

8.10 Week 8 常见错误

- **Decision log 没有 evidence:** 每条 reason 都应引用具体数字 (χ^2 、sigma difference、referee flags)。
- **Medium/high risk 结果被自动标为 accepted:** medium risk 只能标记 pending candidate; high risk 必须生成 approval packet。

- **Codex 一次改太多**：每次只做一个小模块，例如只写 schema 或只写 rules。

8.11 Week 8 导师检查问题

- Analysis Coordinator 是否输出 decision_log.json? 每条理由是否有 evidence?
- 风险等级判定是否合理? medium/high risk 是否触发了对应的处理?
- targeted rerun suggestion 是否结构化 (含 rerun_target、reason、suggested_changes)?
- 学生能否解释 recommendation 和 final accepted result 的区别?

8.12 Week 8 验收标准

- Analysis Coordinator 可以生成 decision_log.json。
- 每个 decision 至少有 2-3 条明确理由。
- 风险等级和 Human Approval Gate 逻辑清楚。
- Analysis Coordinator report 能让导师快速理解结果。
- Snakemake workflow 可以跑到 supervisor decision。
- 学生能解释 accepted result 和 recommendation 的区别。

9 第二个月最终运行命令

第二个月结束时，理想情况下可以用以下命令完整运行：

```
# 1. 运行 workflow dry-run
snakemake -s workflow/Snakefile \
  --configfile configs/l64c64a076_m140.yaml \
  --cores 1 \
  --dry-run

# 2. 正式运行 workflow
snakemake -s workflow/Snakefile \
  --configfile configs/l64c64a076_m140.yaml \
  --cores 1 \
  --rerun-incomplete

# 3. 用 Analysis Engineer 包装运行
python -m lqcd_agent.worker.run \
  --task configs/tasks/run_l64c64a076_analysis_engineer.yaml

# 4. 单独运行 Analysis Coordinator
python -m lqcd_agent.supervisor.run \
  --case outputs/l64c64a076_m140/case_manifest.json \
  --summary outputs/l64c64a076_m140/summary.csv \
  --diagnostics outputs/l64c64a076_m140/fit_strategy_report.json \
  --review outputs/l64c64a076_m140/critical_referee_report.json \
  --decision outputs/l64c64a076_m140/decision_log.json \
  --report outputs/l64c64a076_m140/coordinator_summary.md

# 5. 运行测试
pytest
```

10 第二个月学生需要提交的周报模板

Month 2 / Week X Report

```
1. This week's goal

2. What I implemented
- ...

3. Commands I ran
```bash
...

```



```
...

4. Main outputs
- ...

5. Tests
```bash
pytest
```
Result: pass / fail

6. Main problems and fixes
- Problem:
- Fix:

7. What I learned
- Snakemake / Analysis Engineer / Critical Referee / Analysis Coordinator related
 lesson

8. Questions for advisor
- ...

9. Next week plan
- ...
```

## 11 导师第二个月检查清单

### 11.1 Workflow 检查

- workflow 是否能从 clean outputs 重新生成结果?
- Snakemake rules 的 input/output 是否清楚?
- 是否避免把分析逻辑写进 Snakefile?
- 是否支持 dry-run?

### 11.2 Analysis Engineer 检查

- Analysis Engineer 是否有 task schema?
- 是否有 overwrite protection?
- 是否有 expected output validation?
- 失败时是否生成清楚错误?

### 11.3 Fit Strategy Postdoc、Matrix Element Analyst 和 Critical Referee 检查

- diagnostics 是否结构化?
- reviewer 是否真的在挑问题?
- 是否覆盖 good/bad/ambiguous cases?

### 11.4 Analysis Coordinator 检查

- decision log 是否有明确理由?
- risk level 是否合理?
- medium/high risk 是否要求 Human Approval Gate?
- report 是否足够导师快速阅读?

## 12 第二个月常见失败模式

### 12.1 失败模式 1: Snakefile 里写了太多 Python 分析逻辑

解决方法: Snakefile 只负责调度。复杂分析逻辑放回 `src/lqcd_agent/` 中, 并通过 CLI 调用。

### 12.2 失败模式 2: Analysis Engineer 变成了 Analysis Coordinator

解决方法: Analysis Engineer 只能执行、检查、记录。所有 recommendation 必须放到 Analysis Coordinator。

### 12.3 失败模式 3: Critical Referee 只重复 Fit Strategy Postdoc 的结论

解决方法: Critical Referee 的任务是找风险 (high chi2、model dependence、tmin instability、fit failure rate), 而不是只说 “结果看起来不错”。

### 12.4 失败模式 4: Decision log 没有 evidence

解决方法: 每条 reason 都应该引用 evidence, 例如 chi2、sigma difference、failure rate、reviewer flag。

### 12.5 失败模式 5: Codex 一次改太多

解决方法: 每次 Codex 只做一个小模块, 例如只写 Analysis Engineer schema, 或只写 Analysis Coordinator rules, 不要一次要求完成整个 Month 2。

## 13 Month 2 完成标准

第二个月结束时，学生应能做到：

1. 用 Snakemake 复现 golden example workflow。
2. 用 Analysis Engineer task 文件安全运行 workflow。
3. 从 fit scan summary 中生成 fit diagnostics。
4. 用 Critical Referee 找出潜在风险。
5. 用 rule-based Analysis Coordinator 生成 decision log 和 report。
6. 写出至少 10 个相关 pytest tests。
7. 清楚解释 workflow engine、Analysis Engineer Agent、Fit Strategy Postdoc Agent、Critical Referee Agent、Analysis Coordinator / PI Agent 的区别。
8. 说明为什么当前结果只是 recommendation，而不是未经人工批准的 final accepted result。

### 进入 Month 3 的条件

只有当 Month 2 的 workflow、Analysis Engineer、Fit Strategy Postdoc、Critical Referee、Analysis Coordinator 和 decision log 都能稳定运行后，才适合进入 Month 3 的 LangGraph multi-agent orchestration。否则，直接上 LangGraph 只会把未定义清楚的问题复杂化。